
Documentation: ssh_monitor.py

Overall Purpose: The SSH Login Detective

This module is a specialized detective assigned to guard the most common entrance to a Linux server: the **SSH (Secure Shell) service**. Its one and only job is to detect **brute-force attacks**.

A brute-force attack is the simplest form of hacking. An automated script (a "bot") tries to guess a user's password by attempting to log in over and over again with thousands of common passwords. The tell-tale sign of this attack is a large number of "Failed password" messages in the system's authentication log (/var/log/auth.log), all coming from the same IP address in a short amount of time. This monitor reads that log file, counts the failures, and raises an alarm if an attack is detected.

The Logic: Tailing a File and Handling Rotation

Tailing a "Never-Ending" File

A log file isn't like a normal file that you read once. New lines are constantly being added to the end. It would be very inefficient to re-read the entire file every few seconds. Instead, this monitor "tails" the file.

- **The Bookmark Analogy:** Think of the monitor as reading a book that new pages are always being added to. It uses a "bookmark" (self.log_pos) to remember the exact spot where it last stopped reading.
- On its next check, it opens the book and jumps directly to the bookmark (log.seek(self.log_pos)), then only reads the new pages that have been added. This is an extremely efficient way to process log files.

Handling Log Rotation with Inodes

Linux systems automatically rotate log files to prevent them from growing too large. This means the system will rename `auth.log` to `auth.log.1` and create a brand new, empty `auth.log` file.

- **The Problem:** If our monitor only uses its bookmark, it won't notice that the book has been swapped. It will be stuck looking at its old position in an archived file, completely missing all the new login attempts.
- **The Solution (Inodes):** An **inode** is like a unique serial number that the filesystem gives to every file. When a log file is rotated, the new `auth.log` will have a *different inode number* than the old one.
- This monitor is smart enough to check the file's inode number on every run. If it sees that the inode has changed, it knows a rotation has happened. It then resets its bookmark to the beginning of the new file and clears its memory of old failures.

Code Breakdown

SSHMonitor Class

`__init__(self, ...)` (The Constructor)

- `self.fail_tracker = defaultdict(list)`: This is the detective's notebook. A `defaultdict(list)` is a special kind of dictionary that is very convenient here. If we try to add a failure for a new IP address that isn't in our notebook yet, it automatically creates an empty list for that IP instead of causing an error. The notebook will store data like this: `{'1.2.3.4': [timestamp1, timestamp2], '5.6.7.8': [timestamp3]}`.
- `self.log_inode = None` and `self.log_pos = 0`: This initializes the monitor's memory for the file's "serial number" and its "bookmark" position.

`run(self)`

This is the main loop where the detective does its work.

1. **Check for Log Rotation:** It first gets the `current_inode` of the log file and compares it to the one it has stored. If they are different, it resets its state for the new file.
2. **Read New Log Entries:** It opens the log file, jumps to its saved bookmark (`log.seek(self.log_pos)`), and reads only the new lines.
3. **Find Failures:** For each new line, it uses a regular expression (`re.search`) to check if it's a "Failed password" message. If it is, it extracts the attacker's IP address.
4. **Record Evidence:** It records the event by adding the current timestamp to the list for that IP in its `fail_tracker` notebook.
5. **Update Bookmark:** After reading all new lines, it updates its bookmark to the new end of the file (`self.log_pos = log.tell()`).
6. **Analyze Evidence:** Finally, it calls the `_analyze_failures()` method to see if any of the evidence points to an attack.

`_analyze_failures(self)`

This is where the detective connects the dots.

1. It gets the attack rules (fail_threshold and time_window_minutes) from the config file.
2. It goes through its notebook, looking at one IP address at a time.
3. For each IP, it filters its list of timestamps to count only the recent_attempts that happened within the configured time window.
4. **The Verdict:** It checks if the number of recent attempts is greater than or equal to the threshold.
 - **If YES:** An attack is happening! It logs a high-severity SSH-BRUTE-FORCE alert with all the details. It then **deletes the IP from its notebook** (del self.fail_tracker[ip]) to ensure it doesn't send repeated alerts for the same ongoing attack.
 - **If NO:** It's just a few random failed logins. The monitor does some housekeeping by updating the list of timestamps for that IP to remove the very old ones, keeping the notebook clean.